

■ DOCUMENTACIÓN TÉCNICA

Sistema de Paginación

Arquitectura completa del frontend — Server-side y Client-side pagination en ERP-MVP

📁 volterp_frontend

📅 Julio 2026

🔧 React + TypeScript + Zustand

📄 v1.0

00 Contenido

SECCIÓN	TÍTULO	PÁG
01	Resumen Ejecutivo	2
02	Arquitectura del Sistema — Las 4 Capas	2
03	El Contrato: PagedResult (Backend → Frontend)	3
04	Server-Side Pagination — El patrón principal	3
05	Client-Side Pagination — Cuando el array ya llega completo	4
06	Tabla de Decisión — ¿Cuándo usar cada patrón?	5
07	Casos de Uso Reales en el Código	5
08	Componente Pagination UI — Anatomía	6
09	Features Avanzados — getPageNumbers, dots, auto-hide	7
10	Cobertura de Tests	7

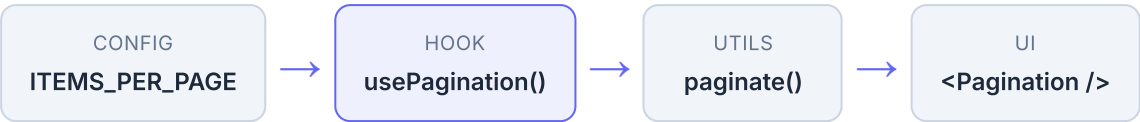
01 Resumen Ejecutivo

🎯 ¿Qué resuelve este sistema?

- **Performance:** Evita renderizar 500+ items en el DOM de una sola vez
- **UX:** Muestra "Mostrando 11-20 de 150" con navegación intuitiva («, <, números, >, »)
- **Escalabilidad:** Funciona igual con 10 items que con 10,000 — el patrón no cambia
- **Reutilización:** Un solo hook, un solo componente, una sola config — aplicados en 12+ páginas

02 Arquitectura del Sistema — Las 4 Capas

El sistema de paginación está separado en 4 capas independientes, cada una con una responsabilidad clara:



CAPA	ARCHIVO	RESPONSABILIDAD	DEPENDENCIA
Config	config/pagination.ts	Valor central: ITEMS_PER_PAGE = 10	Nada
Utils	utils/pagination.ts	Funciones puras: cortar arrays, calcular info, generar números de página	Nada (funciones puras)
Hook	hooks/usePagination.ts	Estado de React: página actual, page size, navegación	Utils
UI	components/UI/Pagination.tsx	Renderizado visual: botones, números, info "Mostrando X-Y de Z"	Utils

03 El Contrato: PagedResult

Todo comienza con el contrato entre el backend (.NET) y el frontend (React). El API devuelve un

`PagedResult<T>`:

```
// domain/types/index.ts
export interface PagedResult<T> {
  rowCount: number; // Total de items en la DB (no solo los de esta página)
  pageNumber: number; // Página actual
  pageSize: number; // Cuántos items por página
  pageCount: number; // Total de páginas
  items: T[]; // Los items de ESTA página (máx 10)
}
```

Este tipo es **genérico** — funciona para Clientes, Productos, Ventas, Compras, o cualquier entidad. El backend siempre devuelve solo los items de la página solicitada.

04 Server-Side Pagination BACKEND

Este es el **patrón principal** del ERP. El backend corta los datos; el frontend solo dice "dame la página X".

Flujo completo — Ejemplo: Clientes

- 1 Usuario navega a página 2 → `usePagination()` actualiza `pageNumber = 2`
- 2 `useEffect` se dispara → llama `fetchClientes(2, 10)`
- 3 `clientService.getClients(2, 10)` → envía `?pageNumber=2&pageSize=10` al API
- 4 Backend ejecuta query → `OFFSET 10 LIMIT 10` → retorna `PagedResult`
- 5 Zustand store actualiza → `clientes = result.items`, `totalCount = result.rowCount`
- 6 `<Table />` renderiza → solo los 10 items de la página 2
- 7 `<Pagination />` → muestra "Mostrando 11-20 de 150" con botones de navegación

```
// 1. Hook maneja el estado de página
const { pageNumber, goToPage, getInfo } = usePagination({
  initialPageSize: ITEMS_PER_PAGE,
});

// 2. useEffect llama al backend cuando cambia la página
useEffect(() => {
  fetchClientes(pageNumber, ITEMS_PER_PAGE);
}, [pageNumber, fetchClientes]);

// 3. El store hace la llamada HTTP con page params
// clienteStore.ts → clientService.getClients(pageNumber, pageSize)
// → GET /api/clients?pageNumber=2&pageSize=10

// 4. El backend devuelve SOLO los items de esa página
// PagedResult: { items: [...10 items], rowCount: 150, pageCount: 15 }

// 5. getInfo calcula la info para el componente UI
const paginationInfo = getInfo(totalCount);

// 6. Render: Table + Pagination
<Table data={paginatedClientes} columns={columns} />
<Pagination pagination={paginationInfo} onPageChange={goToPage} />
```

05 Client-Side Pagination FRONTEND

Cuando el componente recibe el **array completo** como prop (sin API intermedia), el corte lo hace React con `paginate()`.

La diferencia clave

🔦 Server-Side

- El **backend** hace `OFFSET/LIMIT`
- El frontend manda `pageNumber` y `pageSize` como query params
- El API retorna solo 10 items (los de esa página)
- `paginate()` NO se usa — el backend ya cortó
- Necesita `totalCount` del response para el componente UI

💻 Client-Side

- El **frontend** hace `array.slice()`
- No se mandan query params — el array ya llegó completo
- El componente recibe todos los items
- `paginate()` Sí se usa — corta el array localmente
- El total es `array.length`

Ejemplo real — RecentActivities (Dashboard)

El dashboard retorna todas las activities de una. El componente las corta en el front:

```
// RecentActivities.tsx — Client-side pagination
export function RecentActivities({ activities }: Props) {
  // 1. Hook maneja estado (igual que server-side)
  const pagination = usePagination({ initialPageSize: 10 });

  // 2. paginate() CORTA el array en el frontend
  const visibleActivities = paginate(
    activities,           // ← array completo (puede tener 50)
    pagination.pageNumber, // ← página actual
    pagination.pageSize   // ← 10
  );

  // 3. Info para el componente UI
  const info = pagination.getInfo(activities.length);

  return (
    <div>
      {/* Renderiza SOLO los 10 de la página actual */}
      {visibleActivities.map(activity => (
        <ActivityItem key={activity.id} activity={activity} />
      ))}
      {/* Paginación al fondo — se oculta si totalPages ≤ 1 */}
      <Pagination pagination={info} onPageChange={pagination.goToPage} />
    </div>
  );
}
```

💡 ¿Cuándo se usa client-side?

Cuando el endpoint no soporta paginación server-side, o cuando el dataset es pequeño y traer todo es más eficiente que hacer múltiples requests. Ejemplo: las activities del dashboard, el historial de asistencia de un empleado.

Caso híbrido — Clientes con filtro local

Clientes usa **server-side** para la paginación principal, pero **client-side** para el filtro de búsqueda local:

```
// Clientes.tsx - Híbrido: server-side + client-side filtro

// Server-side: fetch del backend con página
useEffect(() => {
  fetchClientes(pageNumber, ITEMS_PER_PAGE); // ← manda page al API
}, [pageNumber]);

// Filtro local: search filtra los items YA cargados
const filteredClientes = useFilter({
  data: clientes, // ← los 10 que ya vinieron del backend
  searchTerm,
  searchFields: (c) => [c.name, c.email, c.phone],
});

// Si el usuario busca, resetea a página 1
// goToPage(1) se llama en el onChange del SearchInput
```

06 Tabla de Decisión

SEÑAL	PATRÓN	¿POR QUÉ?
El endpoint soporta <code>pageNumber</code> / <code>pageSize</code>	SERVER-SIDE	El backend corta → menos datos transferidos → más rápido
El componente recibe un array completo como prop	CLIENT-SIDE	No hay API intermedia → <code>paginate()</code> corta localmente
Dataset > 1000 items	SERVER-SIDE	Traer 1000+ items al front es ineficiente
Dataset < 50 items y no crece	CLIENT-SIDE	Traer todo es más barato que múltiples requests
Se necesita búsqueda/filtro server-side	SERVER-SIDE	El filtro se combina con la paginación en el query
Filtro local sobre datos ya cargados	HÍBRIDO	Server-side para paginar, client-side para filtrar

07 Casos de Uso Reales en el Código

PÁGINA	PATRÓN	USA USEPAGINATION	USA PAGINATE()	USA <PAGINATION />
Clientes	SERVER	✓	✓ (filtro local)	✓
Ventas	SERVER	✓	✗	✓ (en VentasList)
Inventario	SERVER	✓	✗	✓
Compras	SERVER	✓	✗	✓
Contabilidad	SERVER	✓	✗	✓
Proveedores	SERVER	✓	✗	✓
Proyectos	SERVER	✓	✗	✓
RRHH / Empleados	SERVER	✓	✗	✓
Config / Empresas	SERVER	✓	✗	✓
UserManagement	SERVER	✓	✗	✓
SelectorProducto	SERVER	✗	✗	✓
RecentActivities	CLIENT	✓	✓	✓
Asistencia (historial)	CLIENT	✗ (useState)	✗ (slice manual)	✓

⚠️ Nota sobre Asistencia

Asistencia usa `useState` + `slice()` manual en vez de `usePagination()`. Funciona, pero no sigue el patrón estándar del resto del proyecto. Podría refactorizarse para usar el hook.

08 Componente Pagination UI — Anatomía

El componente `<Pagination />` es un componente presentacional puro. No tiene estado interno — todo viene de afuera.

Props

PROP	TIPO	DESCRIPCIÓN
<code>pagination</code>	<code>PaginationInfo</code>	Objeto con: total, page, pageSize, totalPages, hasNext, hasPrev
<code>onPageChange</code>	<code>(page: number) => void</code>	Callback al hacer click en cualquier botón de página

Qué renderiza

```
<div class="pagination">
  <div class="info">
    Mostrando 11 - 20 de 150      ← texto informativo
  </div>
  <div class="controls">
    <button> «« </button>        ← primera página (disabled si hasPrev=false)
    <button> « </button>         ← página anterior
    <button> 1 </button>         ← número de página
    <span> ... </span>           ← separador (si hay gap > 2)
    <button> 2 </button>         ← página actual (active)
    <button> 3 </button>         ← número de página
    <button> » </button>         ← siguiente página
    <button> »» </button>       ← última página (disabled si hasNext=false)
  </div>
</div>
```

Guard: auto-hide

```
if (totalPages ≤ 1) return null; // No renderiza nada si todo cabe en 1 página
```

Este guard es clave: si el backend retorna 8 items y el pageSize es 10, **no se muestra paginación**. El componente se auto-oculta.

Estilos (Pagination.module.css)

BOTÓN NORMAL

Borde sutil, fondo surface

Hover: borde primary + fondo surface-hover

BOTÓN ACTIVO

Fondo primary, texto light

Indica la página actual

BOTÓN DESHABILITADO

opacity: 0.4, cursor: not-allowed

Primera página («« «) o última (» »»)

SEPARADOR DE DOTS

padding: 0 8px, color muted

Aparece cuando hay gap > 2 páginas

09 Features Avanzados

getPageNumbers() — Números inteligentes con ellipsis

Esta función genera la secuencia de números de página que se renderizan, incluyendo separadores "..." cuando hay gaps grandes.

```
// utils/pagination.ts
export function getPageNumbers(currentPage, totalPages): number[] {
  const delta = 2; // Mostrar 2 páginas a cada lado de la actual

  // Paso 1: Recoger páginas candidatas
  // Siempre incluye: página 1, última página, y currentPage ± delta
  for (let i = 1; i ≤ totalPages; i++) {
    if (i === 1 || i === totalPages ||
        (i ≥ currentPage - delta && i ≤ currentPage + delta)) {
      range.push(i);
    }
  }

  // Paso 2: Insertar -1 (representa "...") donde hay gaps
  // Ejemplo: [1, -1, 5, 6, 7, -1, 20]
}
```

Ejemplos de salida

PÁGINA ACTUAL	TOTAL PÁGINAS	SALIDA	VISUAL
1	3	[1, 2, 3]	1 · 2 · 3
1	10	[1, 2, 3, -1, 10]	1 · 2 · 3 · ... · 10
5	10	[1, -1, 3, 4, 5, 6, 7, -1, 10]	1 · ... · 3 · 4 · 5 · 6 · 7 · ... · 10
10	10	[1, -1, 8, 9, 10]	1 · ... · 8 · 9 · 10

🌟 El separador -1

El valor `-1` es un convention interno: en el JSX se renderiza como `<span>...</span>`. No es un número de página — es un placeholder para elipsis visual.

getPaginationInfo() — Info para el componente UI

```
export function getPaginationInfo(total, page, pageSize): PaginationInfo {
  return {
    total,           // 150
    page,           // 2
    pageSize,       // 10
    totalPages: Math.ceil(total / pageSize), // 15
    hasNext: page < totalPages,             // true
    hasPrev: page > 1,                       // true
  };
}
```

getPaginationParams() — Sanitización de inputs

```
export function getPaginationParams(page?, pageSize?): PaginationParams {  
  return {  
    page: page && page > 0 ? page : 1,           // Nunca < 1  
    pageSize: pageSize && pageSize > 0 ? pageSize : 10, // Nunca < 1  
  };  
}
```

10 Cobertura de Tests

El sistema tiene tests unitarios para cada capa:

 PAGINATION.TEST.TS

4 suites · 14 tests

Utils: getPaginationParams, getPaginationInfo, getPageNumbers, paginate

 PAGINATION.TEST.TSX

9 tests

Componente UI: render, disable, click handlers, info text, auto-hide

Casos testeados — Utils

SUITE	TEST	QUÉ VERIFICA
getPaginationParams	Default values	page y pageSize undefined → retorna {1, 10}
	Provided values	page=3, pageSize=25 → retorna {3, 25}
	Invalid values	page=0 o -1 → retorna page=1 (sanitiza)
getPaginationInfo	100 items, page 1	totalPages=10, hasNext=true, hasPrev=false
	100 items, page 5	hasNext=true, hasPrev=true
	100 items, page 10	hasNext=false (última página)
	5 items, page 1	totalPages=1 (auto-hide trigger)
getPageNumbers	3 páginas	[1, 2, 3] — sin dots
	10 págs, current=5	Contiene -1 (dots) + siempre incluye 1 y 10
	Siempre primera y última	array[0]=1, array[last]=totalPages
paginate	Page 1, size 3	[1, 2, 3]
	Page 4, size 3	[10] — última página con 1 item
	Page 10, size 3	[] — página vacía (out of bounds)

Casos testeados — Componente UI

TEST	QUÉ VERIFICA
totalPages ≤ 1 → null	Auto-hide: no renderiza paginación si todo cabe en 1 página
totalPages = 0 → null	Edge case: 0 ítems totales
Render page numbers	Si totalPages > 1, renderiza botones
Prev buttons disabled	En página 1, « y «« están disabled
Next buttons disabled	En última página, » y »» están disabled
First page click	Click en «« → llama onPageChange(1)
Prev page click	Click en « → llama onPageChange(page - 1)
Next page click	Click en » → llama onPageChange(page + 1)
Last page click	Click en »» → llama onPageChange(totalPages)
Info text	Muestra "Mostrando 11 - 20 de 100"